COMPUTER SCIENCE + COMPUTER ENGINEERING FOR GAMES

09

VOLUME NINE

PACKAGE 10 / GRADUATE SPECIALIZATION

Start Next

Graduate entry guide, first session and prerequisite diagnostic

BEGIN HERE

Edition: September 28, 2026

Original lesson sequence preserved. Self-study materials and runnable educational references; not an accredited degree or completed learner submission.

Contents

Start with the claim, not the tool 3

Open these files first . . . 3

Prerequisite gate . . . 3

First 90 minutes . . . 3

Run the portable laboratory . . . 4

What runs here, and what remains learner work . . . 4

Plan the learning work 5

Repeatable lesson cadence . . . 5

Start Gate I early, finish it at the end . . . 5

Entry diagnostic 6

Start with the claim, not the tool

Package 10 expands Volume 9 of Computer Science and Computer Engineering for Games. The original 20 lesson IDs, titles, outcomes, lab briefs and 60 quiz questions are preserved. This is the graduate specialization stage: derive a method, implement it, test it against an independent reference and defend the boundary of the result.

This is a self-study curriculum, not an accredited degree. Reading the handbook or passing the bundled reference tests does not complete the learner's labs or Gate I.

Open these files first

Open COURSE.html after extracting the ZIP. It works offline and includes lesson search, answers hidden until expanded, and local progress export/import. Browser storage can be cleared or restricted, especially for local files; export progress regularly. The PDFs provide a stable print edition. START_NEXT.pdf is this guide; STUDENT_HANDBOOK.pdf is the main text; LAB_WORKBOOK.pdf supplies the execution plan; ANSWER_KEY.pdf is separate; CAPSTONE_SPEC.pdf defines Gate I.

Keep your original curriculum and previous engine/compiler repositories. This package adds depth; it does not renumber the program or replace prerequisite work. Source Markdown/JSON and five standalone module PDFs are included.

Prerequisite gate

Before Module 41, demonstrate Gates B, F and H, or equivalent mathematical, engine and hardware/software reasoning artifacts. You should be comfortable with linear algebra, calculus, probability, native memory ownership, concurrency, rendering, algorithm analysis and reproducible builds. Module-specific dependencies remain printed at each module start.

Take the diagnostic below without the key. A suggested readiness rule is at least eight of ten explained correctly, with every memory-safety and GPU-synchronization misconception repaired. This is a course planning rule, not a standardized credential. Use templates/PREREQUISITE_REPAIR.md, solve a fresh transfer problem, and reassess rather than guessing through a missing prerequisite.

First 90 minutes

1. Spend 15 minutes reading this guide and recording the available environment. Do not install a large graphics stack just to begin.
2. Spend 20 minutes on the diagnostic and identify any prerequisite repair.
3. Spend 15 minutes running the Python reference test suite and saving its output separately from the bundled validation record.
4. Spend 25 minutes beginning Lesson 41.1. Select XPBD 2016 or another bounded original method and write the first claim-evidence row before reading its implementation.
5. Spend 15 minutes implementing the 41.1 warm-up and recording a prediction, failure case and next experiment in the lab journal.

The first session starts the volume; it is not intended to complete a graduate lesson.

Run the portable laboratory

From the extracted package directory:

```
cd code
python3 -m unittest discover -s tests -v
python3 run_experiments.py all --out ../evidence/my_run
python3 check_exercises.py --lesson 41.1
```

Python references use the standard library only. A Python 3.11-or-newer interpreter is the intended baseline; the recorded validation used Python 3.13.5 on Linux x86_64. On Windows, use your installed `py -3` command in place of `python3` as appropriate. The unfinished warm-up intentionally reports PENDING and exits with code 2 until implemented. Do not treat that as a reference-test failure.

C++20/CMake instructions are in code/README.md. No account, API key, paid service or hardware purchase is required to start the portable work. The optional CUDA source and actual GPU project need a supported toolkit/device and their own validation.

What runs here, and what remains learner work

Track	Included executable foundation	Evidence still required
Numerical methods	CSR/PCG, dense oracle, affine fit, bootstrap	Non-diagonal families, derivative checks, fitting/IK extension and independent evaluation
Parallel/GPU	CPU scan/compaction, tiling and scheduling models; native scan; unvalidated CUDA reduction	Actual GPU scan/compaction, synchronization checks, capable baseline and engine integration
Rendering	Small diffuse CPU path tracer, BVH, MIS integral, reservoir selection, scalar inverse fit	Equal-quality studies, dynamic/temporal cases and a bounded rendering reproduction
Engine/simulation	Linear 1D XPBD bar, bounded streaming/store models, finite safety search	Selected system extension, realistic failure schedules and argument-to-implementation mapping
Compilation	CFG interpreter, liveness, pure u32 DSL, folding and C++ source emission	Verifier/analysis extension, legality argument, generated-code inspection and measured integration

CPU analytical models do not satisfy GPU measurement requirements. A non-GPU Gate I track can be pursued while device work is pending, but the pending Module 42 evidence must remain visible. A weighted reservoir is not complete ReSTIR; the bar is not 3D cloth; the DSL is not an operating-system sandbox. The supplied experiments are diagnostic references, not a completed research reproduction.

Plan the learning work

The original estimate remains 550-850 focused hours. That includes assigned reading, derivations, debugging, experiments, reviews and overlapping Gate I integration. The allocations below are planning estimates, not academic credits or deadlines.

Module	Focus	Planning hours
41	Numerical methods and research practice	110-170
42	Parallel and GPU algorithms	110-170
43	Advanced rendering and reconstruction	130-200
44	Engine systems and simulation research	110-170
45	Compilers and graduate artifact review	90-140
Total	Includes the overlapping Gate I project	550-850

At 15 hours/week this is roughly 37-57 weeks; at 25 hours/week, 22-34 weeks. These are arithmetic planning ranges, not a prediction of personal completion time. Reserve time for failed experiments and reviewer revisions rather than assigning every hour to new implementation.

Repeatable lesson cadence

Read the outcome and predict the worked case. Derive the central equation or invariant without code. Read the assigned primary source. Implement the warm-up and then the full lab in your own workspace. Test against an independent reference and an adverse case. Attempt the three original quiz questions and two graduate problems before consulting the key. Record evidence, limitations and review feedback.

A lesson is not complete merely because its visible warm-up passes. A suggested formative standard is all three quiz answers correct after correction, both problems explained, every lab pass condition met, and a fresh transfer case completed without copying the reference. Module project rubrics assess integration beyond recall.

Start Gate I early, finish it at the end

Choose a bounded direction during Module 41. Rendering, simulation, GPU algorithms and compilation are valid depth areas. Use one central project where module work naturally overlaps instead of inventing five unrelated thesis-sized systems. Gate I needs a reproduction, a clearly identified extension or failure analysis, a competent baseline, raw data, an ablation, sensitivity analysis and actual independent review. It prepares Volume 10's thesis; it does not replace it.

Entry diagnostic

Attempt these without the key. Record reasoning and units, not just final numbers.

1. Solve [[2,-1],[-1,2]] x = [1,1], and state why this matrix is SPD.
2. For E(x)=0.5*k*(x-rest)^2, give the force and its units when k is N/m.
3. Under independent finite-variance sampling, what increase in sample count halves standard error?
4. Give the work and span of a balanced binary reduction on 1,024 values.
5. At an assumed sustained 1 GB/s using decimal units, what lower-bound transfer time corresponds to 10 MB, ignoring setup?
6. Why can parallel floating-point sums differ from a serial sum?
7. Why is a callback capturing a destroyed owner unsafe, and name one valid lifetime strategy.
8. Can an area density and a solid-angle density be added directly in an MIS denominator?
9. Why can a global GPU spin barrier that requires all workgroups be deadlock-prone?
10. Does a fixed random seed and a source repository alone guarantee another reader can reproduce a table?

Answers and repair pointers are in ANSWER_KEY.pdf. Use the result to choose review work, not to skip the original prerequisite gates.